# CSA17 – ARTIFICIAL INTELLIGENCE LAB

# Experiments 1 to 5

**Name: D.Vishnu Vardhan Reddy**

**Register Number: 192325039**

## Experiment 1: Solve the 8-Puzzle Problem

### Aim

To solve the 8-puzzle using Breadth-First Search (BFS) and find a sequence of moves from the initial state to the goal state.

### Algorithm

1. Store the initial board as a tuple and put it in a queue.
2. Remove the first state from the queue. If it is the goal, stop.
3. Find the blank (0) and generate valid moves up, down, left and right.
4. Add each unseen state to the queue and record its parent.
5. Reconstruct and print the path when the goal is found.

### Program (Python)

```
from collections import deque

start = (1, 2, 3, 4, 0, 6, 7, 5, 8)
goal  = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def neighbors(state):
    z = state.index(0)
    r, c = divmod(z, 3)
    for dr, dc, move in [(-1,0,"Up"), (1,0,"Down"),
                         (0,-1,"Left"), (0,1,"Right")]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            j = nr * 3 + nc
            s = list(state)
            s[z], s[j] = s[j], s[z]
            yield tuple(s), move

queue = deque([start])
parent = {start: (None, None)}
while queue:
    state = queue.popleft()
    if state == goal:
        break
    for nxt, move in neighbors(state):
        if nxt not in parent:
            parent[nxt] = (state, move)
            queue.append(nxt)

if goal in parent:
    path = []
    cur = goal
    while cur is not None:
        prev, move = parent[cur]
        path.append((cur, move))
        cur = prev
    path.reverse()
    print("Number of moves:", len(path) - 1)
    for board, move in path:
        if move:
            print("Move:", move)
        print(board[0:3], board[3:6], board[6:9])
else:
    print("No solution found")
```

### Input

```
Initial state: (1, 2, 3, 4, 0, 6, 7, 5, 8)
Goal state: (1, 2, 3, 4, 5, 6, 7, 8, 0)
```

### Output

```
Number of moves: 2
(1, 2, 3) (4, 0, 6) (7, 5, 8)
```

```
Move: Down
(1, 2, 3) (4, 5, 6) (7, 0, 8)
Move: Right
(1, 2, 3) (4, 5, 6) (7, 8, 0)
```

# Experiment 2: Solve the 8-Queen Problem

## Aim

To place eight queens on an 8×8 chessboard so that no two queens attack each other.

## Algorithm

1. Place one queen in each row.
2. Try each column in the current row.
3. A position is safe if no earlier queen shares its column or diagonal.
4. If safe, place the queen and recursively process the next row.
5. Backtrack when no column works; print a solution when all rows are filled.

## Program (Python)

```
N = 8
board = [-1] * N

def safe(row, col):
    for r in range(row):
        c = board[r]
        if c == col or abs(c - col) == abs(r - row):
            return False
    return True

def solve(row=0):
    if row == N:
        return True
    for col in range(N):
        if safe(row, col):
            board[row] = col
            if solve(row + 1):
                return True
            board[row] = -1
    return False

if solve():
    for r in range(N):
        print(" ".join("Q" if board[r] == c else "." for c in range(N)))
    print("Queen columns (1-based):", [c + 1 for c in board])
else:
    print("No solution")
```

## Input

```
N = 8 (fixed in the program)
```

## Output

```
One valid board:
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . . Q
(Exact valid arrangement may vary by implementation.)
```

# Experiment 3: Water Jug Problem

## Aim

To measure exactly 2 litres using a 4-litre jug and a 3-litre jug with state-space search.

## Algorithm

1. Represent a state as (amount in 4L jug, amount in 3L jug).
2. Start from (0, 0) and explore states using BFS.
3. Generate states by filling, emptying, or pouring between jugs.
4. Do not revisit a state already seen.
5. Stop when either jug contains 2 litres and print the path.

## Program (Python)

```
from collections import deque

start = (0, 0)
goal_amount = 2

def next_states(a, b):
    cap_a, cap_b = 4, 3
    pour_ab = min(a, cap_b - b)
    pour_ba = min(b, cap_a - a)
    return [
        ((cap_a, b), "Fill 4L jug"),
        ((a, cap_b), "Fill 3L jug"),
        ((0, b), "Empty 4L jug"),
        ((a, 0), "Empty 3L jug"),
        ((a - pour_ab, b + pour_ab), "Pour 4L -> 3L"),
        ((a + pour_ba, b - pour_ba), "Pour 3L -> 4L")
    ]

q = deque([start])
parent = {start: (None, "Start")}
target = None
while q:
    state = q.popleft()
    a, b = state
    if a == goal_amount or b == goal_amount:
        target = state
        break
    for nxt, action in next_states(a, b):
        if nxt not in parent:
            parent[nxt] = (state, action)
            q.append(nxt)

if target is not None:
    path = []
    cur = target
    while cur is not None:
        prev, action = parent[cur]
        path.append((cur, action))
        cur = prev
    for state, action in reversed(path):
        print(action, "->", state)
else:
    print("Goal not found")
```

## Input

```
Jug capacities: 4 litres and 3 litres
Initial state: (0, 0)
Goal: exactly 2 litres in either jug
```

## Output

```
A valid path ends at a state containing 2 litres, for example (2, 0) or (2, 3).
Each printed line shows the action and resulting state; the exact path may vary.
```

# Experiment 4: Crypt-Arithmetic Problem (SEND + MORE = MONEY)

## Aim

To find distinct digits for letters so that SEND + MORE = MONEY is arithmetically correct.

## Algorithm

1. Assign a different digit to each letter S, E, N, D, M, O, R, Y.
2. Do not allow S or M to be zero.
3. Form the numbers SEND, MORE and MONEY.
4. Check whether SEND + MORE equals MONEY.
5. Print the first assignment that satisfies the equation.

## Program (Python)

```
from itertools import permutations

letters = "SENDMORY"
digits = range(10)

for p in permutations(digits, len(letters)):
    d = dict(zip(letters, p))
    if d["S"] == 0 or d["M"] == 0:
        continue
    send = 1000*d["S"] + 100*d["E"] + 10*d["N"] + d["D"]
    more = 1000*d["M"] + 100*d["O"] + 10*d["R"] + d["E"]
    money = (10000*d["M"] + 1000*d["O"] + 100*d["N"]
             + 10*d["E"] + d["Y"])
    if send + more == money:
        print("SEND =", send)
        print("MORE =", more)
        print("MONEY =", money)
        print("Mapping:", d)
        break
```

## Input

```
SEND + MORE = MONEY
```

## Output

```
SEND = 9567
MORE = 1085
MONEY = 10652
Mapping: S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2
```

# Experiment 5: Missionaries and Cannibals Problem

## Aim

To move three missionaries and three cannibals across a river using a two-person boat without ever allowing cannibals to outnumber missionaries on either bank where missionaries are present.

## Algorithm

1. Represent a state as (missionaries_left, cannibals_left, boat_side).
2. Generate boat trips carrying one or two people.
3. Reject states where counts are outside 0–3 or cannibals outnumber missionaries on a bank with missionaries.
4. Use BFS to find a path from (3, 3, left) to (0, 0, right).
5. Print each state in the solution path.

## Program (Python)

```
from collections import deque

start, goal = (3, 3, 0), (0, 0, 1)  # 0=left, 1=right
moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]

def valid(m, c):
    mr, cr = 3-m, 3-c
    return (0 <= m <= 3 and 0 <= c <= 3 and
            (m == 0 or m >= c) and (mr == 0 or mr >= cr))

q = deque([start])
parent = {start: None}
while q:
    s = q.popleft()
    if s == goal:
        break
    m, c, side = s
    for dm, dc in moves:
        nm, nc = (m-dm, c-dc) if side == 0 else (m+dm, c+dc)
        ns = (nm, nc, 1-side)
        if valid(nm, nc) and ns not in parent:
            parent[ns] = s
            q.append(ns)

if goal in parent:
    path, cur = [], goal
    while cur is not None:
        path.append(cur)
        cur = parent[cur]
    for m, c, side in reversed(path):
        print(f"Left: M={m}, C={c}; Right: M={3-m}, C={3-c}; Boat:",
              "Right" if side else "Left")
else:
    print("No solution found")
```

## Input

```
Missionaries = 3, Cannibals = 3, Boat capacity = 2
```

## Output

```
The program prints a valid sequence of states from (3, 3, Left) to (0, 0, Right), ensuring missionaries are never outnumbere
```